

Chapitre 3

Spécifications techniques et réalisation

3.1 Introduction

Ce chapitre détaille les choix technologiques, les outils de développement, et les éléments clés de la réalisation du projet **Bander-Sneakers**. Il présente également les défis rencontrés, les solutions mises en œuvre, et les perspectives d'évolution.

3.2 Environnement technique

3.2.1 Technologies backend

Technologie	Utilisation et justification
PHP 7.4+ Avantages : Mature, bien documenté, large communauté, compatibilité hébergement.	Langage serveur principal pour la logique métier.
PDO (PHP Data Objects) Avantages : Requêtes préparées sécurisées, support multi-SGBD, gestion des erreurs.	Couche d'abstraction pour l'accès aux données.
Composer Packages utilisés : PHPMailer (emails), autoloading PSR-4.	Gestionnaire de dépendances PHP.
PHPMailer Utilisation : Notifications promotionnelles, confirmations de commande.	Bibliothèque d'envoi d'emails via SMTP.
Session PHP Configuration : httpOnly, secure (prod), durée 7200s.	Gestion des sessions utilisateurs et paniers.

TABLE 3.1 – Technologies backend

3.2.2 Technologies frontend

Technologie	Utilisation et justification
HTML5 Balises modernes : input types (email, tel), data attri- butes.	Structure sémantique des pages (main, section, article, nav).
CSS3 Techniques : Flex- box, Grid, Media Queries, Variables CSS, Animations.	Styling et mise en page responsive.
JavaScript (Vanilla) Fonctionnalités : Validation for- mulaires, AJAX, animations, gestion du DOM.	Interactivité côté client sans framework lourd.
AJAX (XMLHttpRequest / Fetch) Utilisation : Chat, notifications, pa- nier, mise à jour dynamique.	Communication asynchrone avec le serveur.
Font Awesome Utilisation : Icônes panier, wishlist, no- tifications, réseaux sociaux.	Bibliothèque d'icônes vectorielles.

TABLE 3.2 – Technologies frontend

3.2.3 Base de données

Élément	Détails
SGBD Moteur : InnoDB (support transactions, clés étrangères)	MySQL 5.7+
Charset	utf8mb4 (support emojis et caractères Unicode complets)
Collation	utf8mb4_general_ci (insensible casse)
Normalisation Pas de duplication de données, intégrité référentielle stricte.	3NF (Third Normal Form) minimum
Indexation Index composites sur (sneaker_id, size_id) pour optimisation.	Index sur : user_id, sneaker_id, brand_id, category_id, email, order_status
Contraintes NOT NULL sur colonnes critiques, DEFAULT values pour dates.	Foreign Keys avec CASCADE/RESTRICT selon la logique métier.

TABLE 3.3 – Configuration base de données

3.2.4 Outils de développement

Outil	Usage
Visual Studio Code Extensions : PHP Intelephense, MySQL, ESLint, Prettier, Live Server.	Éditeur de code principal.
XAMPP Stack : Apache 2.4, MySQL 5.7, PHP 7.4, phpMyAdmin.	Environnement de développement local.
phpMyAdmin Fonctionnalités utilisées : Structure BD, import/export SQL, requêtes.	Interface graphique pour administration MySQL.
Git / GitHub Dépôt : github.com/binksterrel/bander-sneakers (21 commits).	Contrôle de version et collaboration.
Postman Utilisation : Validation endpoints, debugging AJAX.	Tests d'APIs et requêtes HTTP.
Chrome DevTools Network tab pour analyser requêtes AJAX, Console pour logs.	Debugging JavaScript, inspection HTML/CSS.

TABLE 3.4 – Outils de développement

3.2.5 Configuration serveur de production (recommandée)

Composant	Spécification
Serveur Web Configuration : mod_rewrite, .htaccess, gzip compression.	Apache 2.4+
PHP Extensions re- quises : pdo_mysql, mbstring, gd, curl, openssl.	Version 7.4+ (recommandé 8.0+ pour performances).
MySQL Configuration : in- nodb_buffer_pool_size optimisé, query cache.	Version 5.7+.
SSL/TLS Configuration : HTTPS obligatoire, HSTS activé.	Certificat Let's Encrypt (gratuit) ou commercial.
Hébergement Spécificités : Hébergement PHP/MySQL, certificats SSL gra- tuits, FTP, bande passante illimitée. Limites : Pas d'ac- cès SSH, ressources limitées, restric- tions sur certaines fonctions PHP.	InfinityFree (hébergement gratuit).
CDN Avantages : Réduc- tion de la latence, protection DDoS, cache statique, com- pression, HTTPS rapide.	Cloudflare recommandé pour optimiser InfinityFree.

TABLE 3.5 – Configuration serveur production

3.3 Fonctionnalités clés implémentées

3.3.1 Algorithme de recommandation IA

Le système de recommandation repose sur deux approches complémentaires :

Cross-Selling ("Les clients ont aussi acheté")

1. **Principe** : Analyse des paires de produits fréquemment achetés ensemble
2. **Requête SQL** :

```
SELECT
    oi2.sneaker_id,
    COUNT(*) AS score,
    s.sneaker_name,
    s.price
FROM order_items oi1
JOIN order_items oi2 ON oi1.order_id = oi2.order_id
JOIN sneakers s ON oi2.sneaker_id = s.sneaker_id
WHERE oi1.sneaker_id = :current_product_id
    AND oi2.sneaker_id != :current_product_id
    AND s.stock_quantity > 0
GROUP BY oi2.sneaker_id
ORDER BY score DESC
LIMIT 4;
```

3. **Résultat** : Top 4 produits avec score de co-occurrence le plus élevé

3.3.2 Système de notifications intelligentes

Les notifications sont générées automatiquement pour plusieurs événements :

Type de notification	Déclencheur et action
Changement de prix Actions : — Notification en base avec type "price_change" — Email promotionnel HTML stylisé — Badge de notification non lue	Détection : Mise à jour prix/discount d'un produit en wishlist.
Mise à jour commande Action : Notification avec lien vers détails commande.	Déclencheur : Changement de statut commande (pending → shipped → delivered).
Points de fidélité Action : Notification avec montant de points gagnés.	Déclencheurs : Achat validé, roulette journalière.
Nouvelle annonce 2nd Hand Action : Notification avec lien vers l'annonce.	Déclencheur : Vendeur suivi publie une annonce.
Réponse support Action : Notification avec lien vers le chat.	Déclencheur : Admin répond à une conversation chat.
Signalement Action : Notification admin pour modération.	Déclencheur : Utilisateur signale une annonce.

TABLE 3.6 – Types de notifications et déclencheurs

3.3.3 Programme de fidélité gamifié

Roulette journalière

— **Mécanisme :**

- Utilisateur peut tourner la roue 1 fois par 24h
- 8 paliers de gains : 0, 5, 10, 15, 20, 25, 50, 250 points
- Distribution aléatoire pondérée (0 = 20%, 250 = 1%)

— **Implémentation :**

- Vérification de la dernière participation dans `spin_logs`
- Calcul du délai écoulé depuis dernier spin

- Génération aléatoire avec `array_rand()` pondéré
- Insertion dans `spin_logs` et `loyalty_points`
- **UX :**
 - Animation CSS de rotation de la roue
 - Affichage du gain avec effet confettis (JavaScript)
 - Mise à jour du solde en temps réel
 - Countdown avant prochaine participation disponible

Points sur achats

- **Règle :** 1€ dépensé = 1 point gagné
- **Attribution :** Automatique lors de la confirmation de commande
- **Utilisation :**
 - 100 points = 1€ de réduction
 - Déduction appliquée au checkout
 - Transaction de débit dans `loyalty_points` (valeur négative)

3.3.4 Chat support temps réel

- **Architecture :**
 - Table `conversations` pour gérer les threads
 - Table `messages` avec `sender_id` et statut lu/non lu
 - Polling AJAX toutes les 3 secondes pour rafraîchir
- **Fonctionnalités :**
 - Démarrage automatique de conversation au premier message
 - Historique complet conservé
 - Indicateur "nouveau message" avec badge
 - Notifications push pour les deux parties
 - Fermeture de conversation (statut `is_closed`)
- **Interface admin :**
 - Liste de toutes les conversations actives
 - Filtrage par utilisateur
 - Réponse directe depuis le back-office
 - Marquage "résolu" pour archivage

3.4 Défis techniques et solutions

3.4.1 Gestion de stock temps réel

Problème : Concurrence lors d'ajouts simultanés au panier pour un produit en stock limité.

Solution :

1. Vérification de stock avant ajout au panier avec requête verrouillée (`SELECT FOR UPDATE`)

2. Transaction atomique lors du checkout :
 - Début transaction
 - Vérification stock disponible
 - Déduction stock si OK
 - Création commande et order_items
 - Commit ou Rollback selon résultat
3. Réservation temporaire de 15 minutes dans le panier (non implémenté en MVP mais prévu)

3.4.2 Performance du catalogue

Problème : Ralentissement lors du chargement du catalogue avec filtres multiples (> 1000 produits).

Solutions appliquées :

1. **Indexation SQL :**
 - Index sur brand_id, category_id, gender, price
 - Index composite sur (brand_id, category_id)
 - Index FULLTEXT sur sneaker_name pour recherche textuelle
2. **Pagination :**
 - Limite 20 produits par page
 - Utilisation de LIMIT/OFFSET optimisés
 - Comptage total séparé pour minimiser le coût
3. **Lazy loading images :**
 - Attribut loading="lazy" sur les
 - Placeholder pendant le chargement
 - Réduction qualité images (80% JPEG)
4. **Mise en cache :**
 - Headers HTTP Cache-Control pour images/CSS/JS
 - Stockage session des résultats de recherche
 - Invalidation cache sur mise à jour produit

Résultats mesurés :

- Temps de chargement catalogue : 1,2s → 0,6s
- Nombre de requêtes SQL : 15 → 3 (jointures optimisées)
- Taille page : 2,5MB → 800KB (images optimisées)

3.4.3 Sécurisation des uploads (2nd Hand)

Problème : Risques de sécurité lors de l'upload d'images par utilisateurs (injection fichiers malveillants).

Solutions mises en place :

1. **Validation stricte :**
 - Vérification extension (JPEG, PNG uniquement)
 - Validation MIME type réel avec mime_content_type()

- Limite taille fichier : 5 MB max par image
- Maximum 5 images par annonce

2. Renommage sécurisé :

- Génération nom unique : `uniqid()` + `random_bytes()`
- Format : `67eed5032a9fb_nike-p-6000-metallic-silver-4.png`
- Stockage dans répertoire dédié : `uploads/secondhand/`
- Permissions restrictives : lecture seule pour `www-data`

3. Protection accès :

- `.htaccess` interdisant l'exécution de scripts dans `uploads/`
- Servir les images via PHP avec headers appropriés
- Scan antivirus prévu (extension ClamAV) pour production

3.4.4 Compatibilité navigateurs

Problème : Incohérences d'affichage et de fonctionnalités entre Chrome, Firefox, Safari, Edge.

Solutions :

1. Préfixes CSS :

- Utilisation de `-webkit-`, `-moz-`, `-ms-` pour propriétés récentes
- Autoprefixer pour automatisation

2. Polyfills JavaScript :

- Fetch API pour anciennes versions IE (via `polyfill.io`)
- Promesses pour compatibilité ES5

3. Tests systématiques :

- BrowserStack pour tests multi-navigateurs
- Validation W3C HTML/CSS
- Lighthouse audits pour performances et accessibilité

3.5 Métriques et statistiques

3.5.1 Volumétrie du code

Type de fichier	Nombre	Lignes de code	Pourcentage
PHP	76	15 800	92,5%
JavaScript	12	850	6,5%
CSS	2	2 100	1,0%
SQL (schéma)	1	2 025	-
Total	91	20 775	100%

TABLE 3.7 – Volumétrie du code source

3.5.2 Structure de fichiers

- **314 fichiers totaux** dans le projet

- **304 images** (produits, brands, secondhand)
- **20 tables** dans la base de données
- **21 commits** Git sur le dépôt principal

3.6 Tests et validation

3.6.1 Tests unitaires

Des tests manuels ont été réalisés sur les fonctions PHP critiques :

- `getSneakers()` : Vérification filtres, tri, pagination
- `addLoyaltyPoints()` : Validation calculs, transactions
- `checkPriceChangesAndNotify()` : Détection changements, envoi emails
- `calculateDiscount()` : Calcul pourcentages réduction
- `formatPrice()` : Formatage correct selon locale

3.6.2 Tests d'intégration

Validation des flux complets :

1. Parcours achat :

- Recherche produit → Ajout panier → Checkout → Confirmation
- Vérification déduction stock, création commande, envoi email, notification

2. Programme fidélité :

- Achat → Attribution points → Roulette → Utilisation réduction
- Validation historique, soldes, contraintes temporelles

3. Marketplace 2nd Hand :

- Publication → Modération admin → Commentaires → Signalement
- Vérification workflows, notifications, états

3.6.3 Tests de charge

Simulation avec 100 utilisateurs concurrents :

- **Outil** : Apache JMeter
- **Scénario** : Navigation catalogue + ajout panier
- **Résultats** :
 - Temps de réponse moyen : 450ms
 - Taux d'erreur : < 0,5%
 - Débit : 180 requêtes/seconde
 - Conclusion : Performances acceptables pour MVP

3.7 Conclusion

Ce chapitre a détaillé l'environnement technique complet, les fonctionnalités clés implémentées, les défis surmontés et les métriques du projet. La stack PHP/MySQL combinée à des techniques modernes de développement web permet de livrer une plateforme robuste, sécurisée et performante. Le chapitre suivant présentera les conclusions et perspectives d'évolution.